

3637. Trionic Array I

Easy

 Topics

 Companies

 Hint

You are given an integer array `nums` of length `n`.

An array is **trionic** if there exist indices $0 \leq p < q \leq n - 1$ such that:

- `nums[0...p]` is **strictly** increasing,
- `nums[p...q]` is **strictly** decreasing,
- `nums[q...n - 1]` is **strictly** increasing.

Return `true` if `nums` is trionic, otherwise return `false`.

Example 1:

Input: `nums = [1, 3, 5, 4, 2, 6]`

Output: `true`

Explanation:

Pick `p = 2`, `q = 4`:

- `nums[0...2] = [1, 3, 5]` is strictly increasing (`1 < 3 < 5`).
- `nums[2...4] = [5, 4, 2]` is strictly decreasing (`5 > 4 > 2`).
- `nums[4...5] = [2, 6]` is strictly increasing (`2 < 6`).

Example 2:

Input: `nums = [2, 1, 3]`

Output: `false`

Explanation:

There is no way to pick `p` and `q` to form the required three segments.

Constraints:

- `3 <= n <= 100`
- `-1000 <= nums[i] <= 1000`

Python:

class Solution:

def isTrionic(self, nums: List[int]) -> bool:

n = len(nums)

peak = -1

valley = n

```

for i in range(n - 1):
    if peak == -1 and nums[i] >= nums[i + 1]:
        peak = i
    if valley == n and nums[-1 - i] <= nums[-2 - i]:
        valley = n - 1 - i
    if peak != -1 and valley != n:
        break

```

```

if peak <= 0 or valley >= n - 1:
    return False

```

```

for i in range(peak, valley):
    if nums[i] <= nums[i + 1]:
        return False

```

```

return True

```

JavaScript:

```

const isTrionic = nums => {
    const n = nums.length;
    let peak = -1;
    let valley = n;

    for (let i = 0; i < n - 1; i++) {
        if (peak === -1 && nums[i] >= nums[i + 1])
            peak = i;

        if (valley === n && nums.at(-1 - i) <= nums.at(-2 - i))
            valley = n - 1 - i;

        if (peak !== -1 && valley !== n)
            break;
    }

    if (peak <= 0 || valley >= n - 1)
        return false;

    for (let i = peak; i < valley; i++)
        if (nums[i] <= nums[i + 1])
            return false;

    return true;
};

```

Java:

```
class Solution {
    public boolean isTrionic(int[] nums) {
        int n = nums.length;
        int peak = -1;
        int valley = n;

        for (int i = 0; i < n - 1; i++) {
            if (peak == -1 && nums[i] >= nums[i + 1])
                peak = i;

            if (valley == n && nums[n - 1 - i] <= nums[n - 2 - i])
                valley = n - 1 - i;

            if (peak > -1 && valley < n) break;
        }

        if (peak < 1 || valley > n - 2)
            return false;

        for (int i = peak; i < valley; i++)
            if (nums[i] <= nums[i + 1])
                return false;

        return true;
    }
}
```